

Build the harness. Then test the models.

- I approached the brief as a harness problem: connect model APIs to a controlled Slack environment.
- I needed to understand how agents observe a page, act, and know when a task is complete.

01 · RESEARCH



How can agents act?

Screenshots, page controls, and APIs

02 · BUILD



How can I test them?

Reset, observe, act, check, record

03 · COMPARE



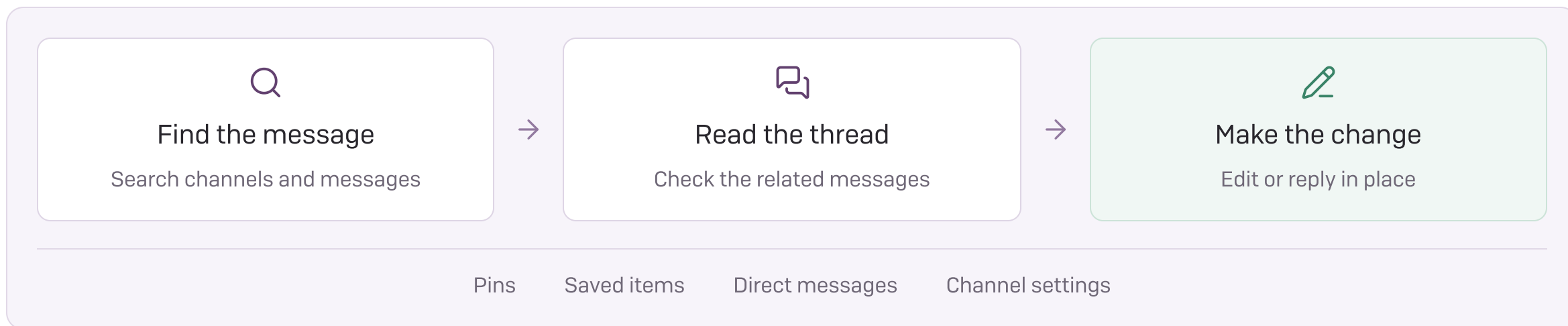
Which models work?

Task outcomes, time, cost, and traces

-
- Kevin Liu · CUA implementation exercise · October 2026 · Based on the supplied assignment brief.

Build a small set of complete workflows

- I built the app with fictional people, channels, and messages.
- I chose search, threads, and message editing as the core workflows.
- People and browser agents use the same interface.



- [React UI](#) • [SQLite state](#) • [Try the workspace without a key](#) • [Prior-work review](#).

18 tasks use the full workspace

Six focused tasks

Update a topic

Channel → edit topic → save

Reply in a thread

Find request → open thread → reply

Edit a message

Find own message → edit → save

Triage an incident

Find active incident → react → pin

Delete a draft

Find own draft → delete → confirm

Send a handoff

Read current details → compose DM

Coordinate work

Coordinate a release *

Read sources → edit → reply → topic → DM

Close out an incident

Read resolution → replace pin → reply → topic

Refresh saved work

Find current sources → replace saved items → DM

Record a design decision

Read approval → react → pin → save → details

Repair an outdated handoff

Read correction → edit DM → react → save

Complete a QA sign-off

Read test results → reply → fix reaction → save

Combine several changes

Publish a release update

Read source → format update → pin → delete draft

Brief the on-call engineer

Read rotation → DM primary → save → details

Correct a threaded estimate

Read review → edit own reply → react → reply

Refresh the launch board

Find plan → replace pin → save → topic

Hand off an approved design *

Read decision → react → DM → reply → save

Prepare a release retrospective

Read three sources → summarize → pin → save → DM

- These diagrams summarize the intended workflows. Exact instructions and preservation checks remain in each task.
- * Two original tasks contain wording problems. Their results remain unchanged and are discussed later.

Check the final workspace state

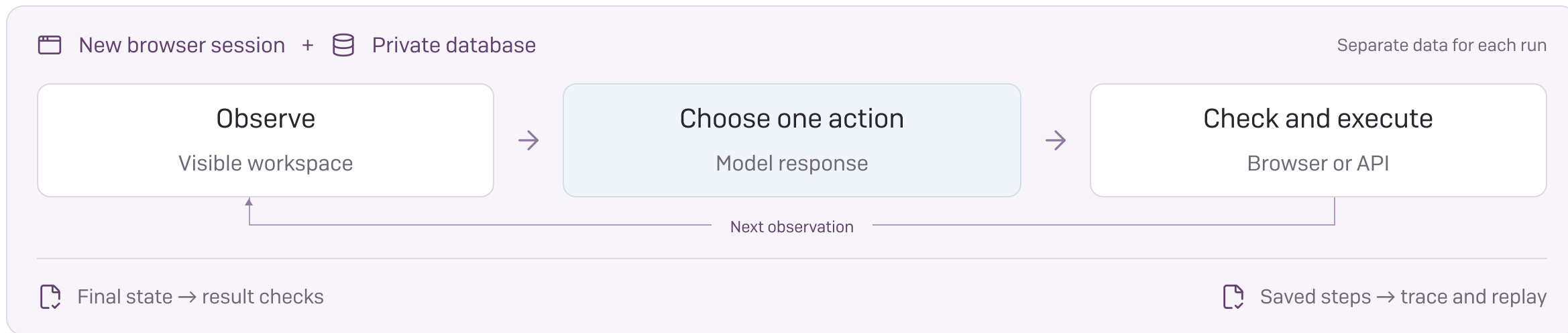
- Workspace state is the stored data: messages, threads, topics, and saved items.
- The checker compares this data with the task requirements.
- A model's claim that it finished does not affect the result.

 **Example: update a channel topic**
Set the requested topic. Make no other changes.

 Correct topic Exact text in channel settings	 No extra changes Other workspace data stays the same	Both pass Reward = 1
--	---	--------------------------------

- The checker grades the final state. It does not penalize a wrong action that the agent fully undoes.
- [Task definitions and result checks.](#)

Start each run in a fresh workspace



- The harness is the code that manages each run.
 - It resets the workspace, sends observations, executes actions, and records results.
 - Each run has its own browser session and database.
-
- Playwright controls the browser. Separate data does not protect the host from malicious code.

I then used the harness to compare models

- The same run process worked with different models.
- I used it to compare 17 models on 18 tasks.
- This expanded the project into a benchmark prototype.

Original assignment



Working environment

Focused Slack workflows
Known starting state



Repeatable harness

Reset · act · check · record
Select a model



Benchmark prototype

18 tasks × 17 models
One attempt per pair

The comparison also needs clear tasks and consistent test conditions.

Each interface gives the agent different controls

Interface	What the agent receives and uses	Earlier passes / attempts
 Pixels	Screenshot → mouse and keyboard	0 / 1
 Accessibility	Named controls → element actions	2 / 6
 Page JSON	Controls as JSON → element actions	2 / 7
 API	Workspace data → direct changes	4 / 6

- Earlier runs used different tasks and models, so they cannot show which interface works best.
- The larger comparison uses only accessibility controls.

- 20 attempted: 8 passed, 11 incomplete, 1 blocked. 10 planned runs did not start.
- 126 requests · \$0.149874 estimated allowance, including missing-usage reservations · [Earlier interface evidence](#).

Test the software and the agents separately

APP BEHAVIOR



174 backend tests
91 browser tests

Does the software work?

RESULT CHECKS



84 valid states
2,583 invalid states

Does grading reject wrong results?

AGENT BEHAVIOR



306 recorded runs

Can the agent complete the task?

- Scripted browser tests confirm that the app supports each workflow.
- Known valid and invalid states test the checker.
- Neither check proves that a new reader can understand the task.

-
- [Software verification](#) • Model evidence remains separate from scripted tests.

Run each task once with each model

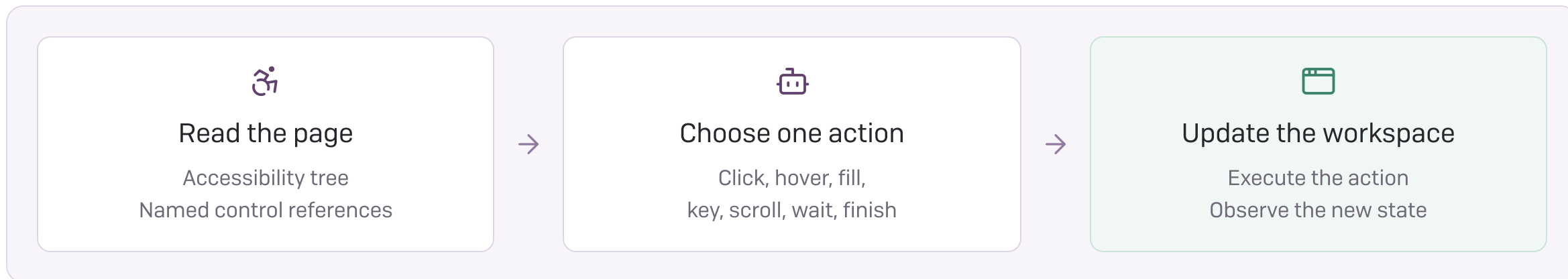
$$\begin{array}{ccccccc} \text{🏠} & & \text{☑️} & & \text{🛡️} & & \\ 17 & \times & 18 & \times & 1 & = & 306 \\ \text{model routes} & & \text{tasks} & & \text{attempt per pair} & & \text{recorded attempts} \end{array}$$

- I used six simple tasks and 12 tasks with several steps.
- All models use the same task data, action format, and result checks.
- One run per pair cannot show how often a model will succeed.

40 actions 180 s per run \$5 estimated allowance per run

- I kept 105 earlier runs and added 201 new runs. I did not replace failed runs.
- [Node 24 setup and API key instructions](#) · [Exact routes, limits, and run history](#).

All 306 runs used accessibility controls



- All runs used seed 1042, the last four turns, and no site guide.
- All runs allowed 40 actions, 180 seconds, and 4,096 output tokens per request.
- These results compare model routes under one interface, not which interface works best.

-
- Checked against all 306 hash-verified episode records. Reasoning used provider defaults; routing and collection time were not controlled.
 - No screenshot-only condition, direct API condition, or llms.txt comparison is included in these 306 runs.

Results for 17 models

1 per task/model · Collection complete

306 / 306 attempted · 152 passed · 61 incomplete · 93 blocked

\$189.3110

Usage estimate

\$3.7794

Unresolved · 35 calls

\$193.0904

Total allowance · 306 trials

Model ↕	Trials ↕	Passed ↕	Incomplete ↕	Blocked ↕	Median time ↕	Usage estimate ↕	Unresolved ↕	Total allowance ↕
 GPT-6 Astra	18 / 18	14 / 18	1	3	27.1s	\$35.9623	\$0.0000	\$35.9623
 GPT-6.1 Sol	18 / 18	14 / 18	4	0	28.5s	\$8.4352	\$0.0000	\$8.4352
 GPT-6 Luna	18 / 18	9 / 18	8	1	39.4s	\$0.5394	\$0.0092	\$0.5486
 GPT-OSS 120B	18 / 18	5 / 18	2	11	180.1s	\$0.6682	\$0.1358	\$0.8040
 Claude Fable 5.1	18 / 18	13 / 18	1	4	62.8s	\$48.4714	\$0.0000	\$48.4714
 Claude Opus 5.5	18 / 18	13 / 18	4	1	38.3s	\$28.6987	\$0.3741	\$29.0728
 Claude Sonnet 5.5	18 / 18	7 / 18	4	7	39.9s	\$13.9323	\$0.3304	\$14.2627
 Claude Haiku 4.5	18 / 18	6 / 18	3	9	28.3s	\$3.7921	\$0.0000	\$3.7921
 Grok 4.7	18 / 18	10 / 18	6	2	34.3s	\$11.9101	\$0.3809	\$12.2910
 Qwen 3.8 Max	18 / 18	7 / 18	1	10	125.1s	\$6.9999	\$1.0639	\$8.0638
 DeepSeek V4 Pro	18 / 18	8 / 18	1	9	106.8s	\$4.6816	\$0.2547	\$4.9363
 DeepSeek V4.1 Flash	18 / 18	7 / 18	3	8	65.9s	\$1.3825	\$0.0541	\$1.4365
 GLM 5.3	18 / 18	7 / 18	0	11	125.3s	\$3.6307	\$0.3453	\$3.9760
 GLM 5.3 Flash	18 / 18	8 / 18	0	10	46.2s	\$0.4444	\$0.0000	\$0.4444
 Kimi K3	18 / 18	11 / 18	3	4	50.0s	\$14.0051	\$0.7958	\$14.8009
 MiniMax M3	18 / 18	5 / 18	10	3	25.9s	\$1.4698	\$0.0353	\$1.5052
 Nemotron 3 Ultra	18 / 18	8 / 18	10	0	50.1s	\$4.2874	\$0.0000	\$4.2874

Passed

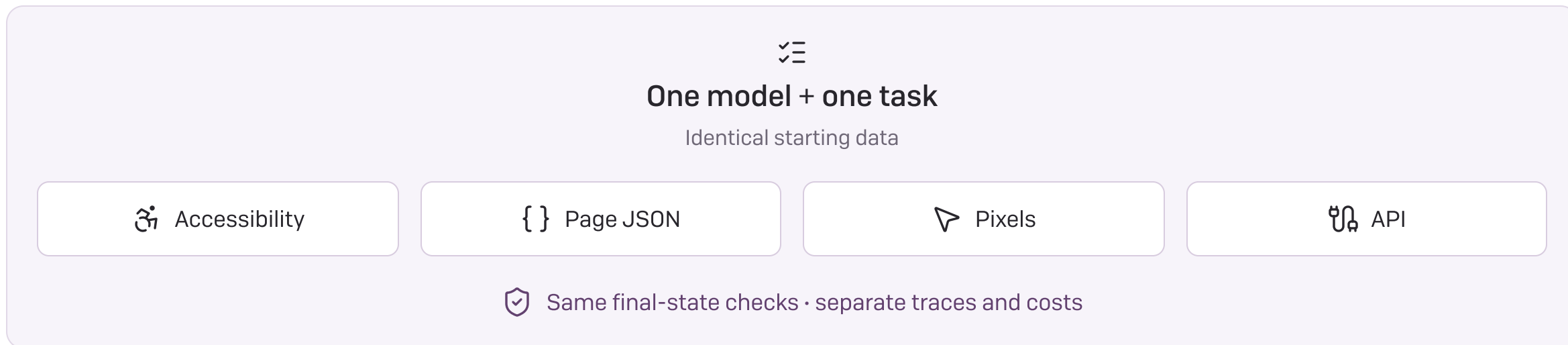
Incomplete

Blocked

Not run

- 18 public tasks. 1 run per task and model. All use accessibility controls. Median time includes blocked runs.
- Total allowance adds the usage estimate to money reserved for missing usage. These are estimates, not invoices. Including earlier tests: \$194.1058 of the \$300 budget. [Full costs CSV](#) · [JSON](#)
- 105 earlier runs are included. No retries. One run per task cannot establish a reliable model ranking. Two tasks have unclear instructions. [Read the failure analysis](#).
- [Test plan](#) · [Recorded runs](#) · [Download all trials \(CSV\)](#) · [Earlier 36-trial pilot: 20 passed](#)

Change the interface, keep the task fixed



- Four models attempt six tasks through four interfaces, giving 96 planned runs.
- Each attempt starts fresh, and the interface order is randomized within each model and task.
- The study uses a separate \$25 allowance with no retries.

-
- Sol, Sonnet, Qwen and Grok • topic, thread reply, message edit, incident closeout, saved cleanup, decision record.
 - Seed 2042 • low reasoning • recent-four history • no guide • 40 actions • 180 seconds • \$1 ceiling per cell.

Interface comparison: 96 recorded attempts

Passed / attempted • six tasks per cell • click headers to sort

Model ↕	Accessibility ↕	Page JSON ↕	Pixels ↕	Actor API ↕	Allowance ↕
 GPT-6.1 Sol	<u>6 / 6</u> 0 incomplete • 0 blocked	<u>5 / 6</u> 0 incomplete • 1 blocked	<u>1 / 6</u> 0 incomplete • 5 blocked	<u>6 / 6</u> 0 incomplete • 0 blocked	\$3.9430
 Claude Sonnet 5.5	<u>4 / 6</u> 0 incomplete • 2 blocked	<u>4 / 6</u> 1 incomplete • 1 blocked	<u>2 / 6</u> 1 incomplete • 3 blocked	<u>5 / 6</u> 1 incomplete • 0 blocked	\$7.2784
 Qwen 3.8 Max	<u>5 / 6</u> 1 incomplete • 0 blocked	<u>4 / 6</u> 1 incomplete • 1 blocked	<u>0 / 6</u> 0 incomplete • 6 blocked	<u>5 / 6</u> 0 incomplete • 1 blocked	\$5.4829
 Grok 4.7	<u>3 / 6</u> 1 incomplete • 2 blocked	<u>3 / 6</u> 3 incomplete • 0 blocked	<u>0 / 6</u> 1 incomplete • 5 blocked	<u>5 / 6</u> 1 incomplete • 0 blocked	\$6.0573

[All traces and replays ↗](#) [Costs CSV ↓](#) [Paired results and timing ↗](#)

- API passed 21/24 attempts, accessibility passed 18/24, Page JSON passed 16/24, and pixels passed 3/24.
- 19 pixel runs were blocked, including 14 connection failures.
- API measures tool use, and one attempt per condition cannot establish a reliable winner.

• Verified snapshot: 58 passed • 11 incomplete • 27 blocked • 0 not run. Usage estimate \$20.7180 + unresolved \$2.0435 = \$22.7616 of \$25. Updated 2026-10-05 07:42 UTC.

• Pixels use low-detail screenshots. Interfaces differ in visible information and action size, not just formatting.

The checks and the instructions must agree

- I tested the checker, but I did not ask an independent reader to solve every task.
- Two tasks had unclear instructions, so their scores do not measure model ability alone.

TASK TEXT

“DESIGN”

No instruction to replace this word

RESULT CHECK

≠

“Willow”

Checker requires this exact name

Release sync

Unclear source location
0 / 17 passed

Retrospective

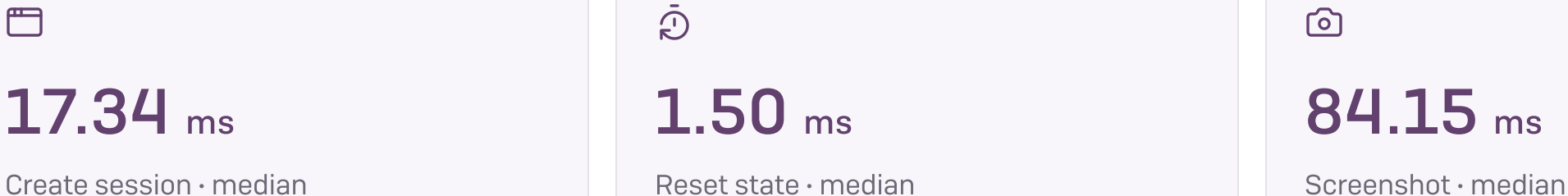
Every run reached a limit
0 / 17 passed

Handoff repair

Sol and Astra clicked Cancel
5 / 17 passed

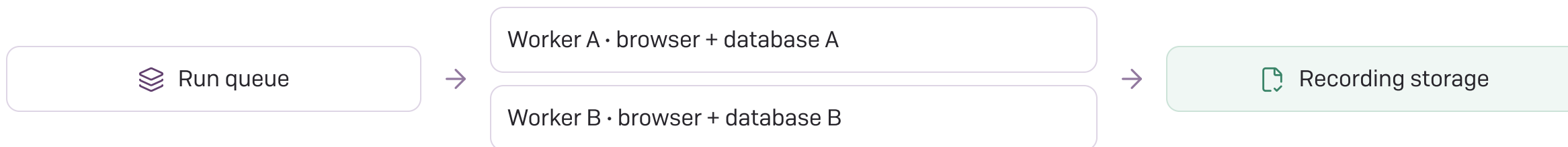
- Six completed runs copied DESIGN. The task did not explain that this word was a placeholder.
- Best individual models: 14 / 18 tasks. Across all models: 15 / 18 tasks passed at least once.
- Recorded actions and failure analysis. Keep all original outcomes. A wording fix requires a new task version.

Measure local speed before adding more workers



- These timings exclude model calls and do not measure how many agents can run at once.
- Next, measure memory use, completed runs per minute, and cleanup with several workers.

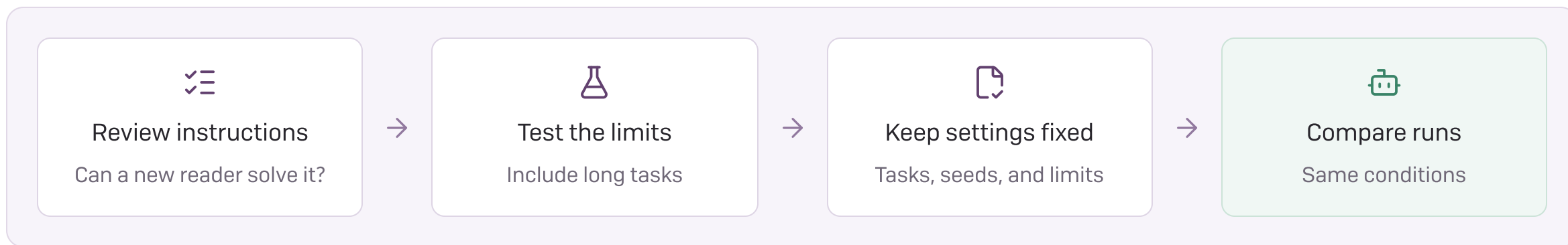
PROPOSED DESIGN



- Apple M5 Max • Node 24 • 100 sessions • 400 reads at concurrency 16 • 10 sequential browser sessions.
- Timing samples and test setup. The multi-worker design is proposed, not measured.

Validate each task before the next comparison

- Ask someone outside the project to solve each task from its instructions.
- Compare their interpretation with the expected result.
- Set run limits with both short and long tasks.
- Use new task variants and repeat runs to measure reliability.



Validate the task instructions as carefully as the grading code.

- [Detailed engineering review](#) · [Remaining validation work](#). These are next steps, not completed results.

Use the app and inspect a recorded run



01 · TRY

Use the workspace

Search, open a thread, edit



02 · COMPARE

Choose a model and task

All 306 attempts and costs



03 · REVIEW

Inspect the run

Actions, changes, result checks

- Which workflows should the next version test?
- Should the score include wrong actions that the agent later undoes?
- How many agents must run at once, and what should each run cost?

-
- Relay can reset, execute actions, and return a reward. No reinforcement learning training has been run.
 - [Code and setup](#) · [Requirements and remaining work](#).